



Embedding Flaris in a C application

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.4.0 **Generated** 2026-09-29 **License** see repository

© 2026 SolidInvention AB — created and maintained by SolidInvention AB

Flaris ships as a library as well as a program. Link `libflaris` into your application and Flaris becomes its scripting layer: load scripts, call their functions from C, and drive the scheduler from your own loop.

This is the counterpart to the [FFI guide](#). FFI is *a script reaching a shared library you did not compile against*; this is *your application and a script calling each other directly*, in the same process, with nothing copied between them.

- 1. What you download
- 2. Hello, host
- 3. Shipping scripts with your application
- 4. Lifecycle
- 5. Configuration
- 6. Loading code
- 6a. Contexts
- 7. Calling into Flaris
- 8. Values and ownership
- 9. Native modules: your C functions, called from script
- 10. Restricting what a script can reach
- 10a. Signals from the script
- 10b. Stopping a running script
- 10c. Embedding from C#
- 11. Driving the scheduler
- 12. Errors and diagnostics
- 12a. Threads: one VM per thread
- 13. Limits and what is not supported
- 14. API reference

1. What you download

Get the library for your platform from the [downloads page](#). Each archive unpacks to the same shape:

```
flaris-lib/
├─ include/
│   └─ flaris.h      # the embedding API – the only header you include
└─ lib/              # the static and shared library
```

One header, and it is self-contained: it declares the whole API in terms of C types, so nothing about the VM's internals leaks into your build. Values cross the boundary as `FlarisValue`, an opaque one-word handle.

Platform	Static	Shared
macOS ARM64	libflaris.a	libflaris.dylib
Linux x86_64 / ARM64	libflaris.a	libflaris.so
Windows x64	libflaris.a	flaris.dll + libflaris.dll.a

This is the same VM the `flarism` command runs — compiler, runtime, JIT, standard builtins and debugger — without the command-line front end. Nothing a script does, and nothing a bad input file contains, makes it call `exit()`; your application stays in control. Section 12 states the guarantee exactly.

Include path and link flags. Point your compiler at `include/`, link the library, and add the system libraries the VM needs:

```
# macOS
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lm -framework Security -framework CoreFoundation

# Linux
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lm -ldl -lpthread

# Windows (MSYS2 clang64)
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lws2_32 -lbcrypt -liphlpapi -lsecur32 -lcrypt32
```

Linking the shared library instead. The shared library exports exactly what this document describes - the `Flaris*` functions listed in section 13, plus `flarisConfigDefaults` - and nothing else; the VM's internals are hidden, so no future release can break you by moving one. On Windows, add `-DFLARIS_DLL` when you build against `flaris.dll`, so the declarations become `dllimport`:

```
cc host.c -DFLARIS_DLL flaris-lib/lib/libflaris.dll.a -Iflaris-lib/include -o host
```

macOS and Linux need no define, but they do need an `rpath`, or the executable will not find the library beside it at launch:

```
# macOS
cc host.c flaris-lib/lib/libflaris.dylib -Iflaris-lib/include \
    -Wl,-rpath,@executable_path -o host

# Linux
cc host.c flaris-lib/lib/libflaris.so -Iflaris-lib/include \
    -Wl,-rpath,'$ORIGIN' -o host
```

2. Hello, host

Complete and compilable — nothing is elided.

```
#include "flaris.h"
#include <stdio.h>

static const char *SCRIPT =
    "fn Greet(who: string): string {\n"
    "    return \"hello, \" + who;\n"
    "}\n";

int main(void)
{
    if (FlarisInitVM(NULL) != FLARIS_OK)
        return 1;

    FlarisContext *ctx;
    if (FlarisCreateContext(NULL, &ctx) != FLARIS_OK) {
        FlarisShutdownVM(1);
        return 1;
    }

    if (FlarisLoadSourceText(ctx, SCRIPT, "greet.flr") != FLARIS_OK) {
        FlarisShutdownVM(1);
        return 1;
    }

    FlarisValue args[1] = { FlarisString("world") };
    FlarisValue result;
    char err[256];

    int rc = FlarisCall(ctx, "Greet", args, 1, &result, err, sizeof err);
    if (rc == FLARIS_OK) {
        size_t len = 0;
        const char *text = FlarisAsString(result, &len);
        printf("%.s\n", (int)len, text);
        FlarisReleaseValue(result);
    } else {
        printf("error: %s\n", err);
    }

    FlarisReleaseValue(args[0]); /* FlarisCall borrows; you still own it */
    FlarisShutdownVM(0);         /* destroys any context you left behind */
    return 0;
}
```

```
$ cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lm -framework Security -framework CoreFoundation
```

```
$ ./host  
hello, world
```

Four things in that program are worth naming now, because everything else builds on them.

A **context** is one script's own global environment. Scripts are loaded into a context and called through it, and what one defines is invisible to every other — two scripts that both declare `config` no longer overwrite each other. Section 6a is about them.

`FlarisValue` is an **opaque handle** to a value living inside the VM. It is a single word; you never dereference it, and every read goes through a function like `FlarisAsString`. Nothing is copied when one crosses the boundary.

`FlarisCall` **borrow**s its arguments — you built them, you release them. The **result is yours**, and `FlarisReleaseValue` is how you drop it.

Every entry point returns a status code rather than aborting. A script that throws, a file that will not compile, a corrupt `.flx`, a module whose pinned hash does not match and a function that does not exist are all ordinary return values.

3. Shipping scripts with your application

You can hand Flaris either source (`.fls`) or compiled bytecode (`.flx`). For anything you ship, prefer bytecode:

- it does not expose your script source,
- it skips compilation at startup,
- it is validated on load, so a corrupt or mismatched file is refused rather than half-run.

Compile with the `flarism` toolchain as part of your build:

```
flarism --compile game.flx game.flx
```

Then load it at runtime:

```
FlarisLoadBytecode(ctx, "game.flx");
```

A script you load this way is a **library of functions**, not a program. Give it either an `export` list or a `Main`, because `--compile` requires one of the two:

```
fn Greet(who: string): string { return "hello, " + who; }  
fn Update(dt: float): int     { /* ... */ return 0; }  
  
export { Greet, Update };
```

Everything the script declares is callable by name afterwards, whether or not it appears in the `export` list — the list is there to satisfy the compiler and to document intent.

Bytecode is portable across platforms and machines: a `.flx` built on one target runs on any other.

4. Lifecycle

```

FlarisInitVM(&cfg)
|
|─ FlarisCreateContext(&opt, &ctx)  ← one per script; as many as you like
|   |
|   |─ FlarisLoadBytecode(ctx, "game.flx")  ← or FlarisLoadSource(...)
|   |─ FlarisCall(ctx, "Update", ...)      ← as often as you like
|   |
|   └─ FlarisDestroyContext(ctx)  ← optional; shutdown reclaims it
|
|─ FlarisPump(NULL, 0)              ← if a script uses timers/fibers/I0
|
FlarisShutdownVM(0)

```

`FlarisShutdownVM` releases everything — including any context you did not destroy yourself — and leaves your process able to start a fresh VM. It is idempotent: calling it twice, or without a matching init, does nothing.

Reloading **one** script no longer means restarting the VM: destroy its context and create a new one.

5. Configuration

Pass `NULL` to `FlarisInitVM` for defaults, or fill in a `FlarisConfig`:

```

FlarisConfig cfg = flarisConfigDefaults;
cfg.installSignals = FLARIS_OFF;          /* keep your own signal handlers */
cfg.startIoPool    = FLARIS_OFF;          /* stay single-threaded          */
cfg.grantCaps      = FLARIS_CAP_UNSAFE;   /* the script may use Ffi/Memory */
cfg.maxFibers      = 64;                  /* power of two                  */
cfg.stackSize      = 8192;
cfg.libsPath       = "/opt/myapp/scripts";

if (FlarisInitVM(&cfg) != FLARIS_OK) {
    /* a value was rejected – nothing was initialised */
}

```

Switches

Every switch is tri-state: `FLARIS_DEFAULT` (which is `0`, so a zeroed struct is a valid starting point), `FLARIS_ON`, or `FLARIS_OFF`. Anything else is rejected rather than read as a default.

Switch	Default	What it does
<code>installSignals</code>	on	Installs the VM's handlers for <code>SIGINT</code> , <code>SIGTERM</code> , <code>SIGHUP</code> , <code>SIGUSR1</code> and <code>SIGUSR2</code> (and ignores <code>SIGPIPE</code>), so a script can receive them through <code>VM.OnSignal</code> . A signal no script handler has claimed ends the process at once with exit code 128 + signal number, nothing torn down. Off leaves your handlers alone. The VM installs no <code>SIGSEGV</code> handler either way — a fault inside it reaches <i>you</i> .
<code>startIoPool</code>	on	Starts the I/O worker threads. Off keeps your process single-threaded; I/O still completes, inline on the VM thread.
<code>jit</code>	on	Native code generation.
<code>optimizations</code>	on	Constant folding and peephole passes.
<code>debugInfo</code>	off	Keeps source line numbers, so stack traces report real lines instead of line 0. Function and file names are kept either way.
<code>signatureChecks</code>	on	Verifies a <code>.flx</code> fingerprint when one is pinned.
<code>requireSigned</code>	off	Refuses <code>.flx</code> not signed by a trusted key.
<code>verbose</code>	off	Emits the VM's own progress chatter at <code>FLARIS_LOG_INFO</code> .
<code>colors</code>	auto	ANSI colour in diagnostics. Irrelevant once you set a log handler.

Capabilities

What a script may *do* is a bitmask rather than a switch, set with `cfg.grantCaps` and `cfg.denyCaps` (deny is applied after grant, so a bit in both is withheld):

Capability	Default	What it allows
<code>FLARIS_CAP_UNSAFE</code>	off	<code>Ffi.*</code> , raw <code>Memory</code> access and <code>Buffer</code> addresses. Implies <code>FLARIS_CAP_FFI</code> .
<code>FLARIS_CAP_FFI</code>	off	<code>Ffi.*</code> alone, without the rest of <code>UNSAFE</code> .
<code>FLARIS_CAP_IMPORT_LOCAL</code>	on	<code>import</code> may resolve from disk and <code>libsPath</code> .
<code>FLARIS_CAP_IMPORT_REMOTE</code>	off	<code>import</code> may fetch over <code>https://</code> .
<code>FLARIS_CAP_IMPORT_INSECURE</code>	off	...and over plain <code>http://</code> too. Needs <code>REMOTE</code> as well.

FLARIS_CAP_UNSAFE is a trust decision. It lets a script load native code into your process via `Ffi` and read or write raw addresses via `Memory`. Leave it off for scripts you do not control.

The import capabilities are the other one. A script that may fetch and run code from a URL is only as trustworthy as that URL. The default is local imports only; the `flarism` command grants `IMPORT_REMOTE` itself, because it runs what the user chose to run.

Limits

Limit	Default	Range
<code>stackSize</code>	4096	64–65535 value-stack slots
<code>fifoSizeCfg</code>	1024	power of two, 8–1024
<code>maxSlabs</code>	1024	object-pool ceiling
<code>maxFibers</code>	256	power of two, 2–1024
<code>maxFrames</code>	64	8–1024 call frames per fiber
<code>ioThreads</code>	0	0 scales to the CPU count; ≤ 16
<code>libsPath</code>	none	Where <code>import</code> looks, ; -separated. Borrowed — must outlive the VM.

`stackSize`, `maxFrames` and the scheduling quantum can additionally be narrowed per context — see section 6a.

Every limit reads `0` as "keep the default", so you set only what you care about. An out-of-range value makes `FlarisInitVM` **fail** rather than being quietly clamped, and nothing is initialised when it does.

6. Loading code

Code is loaded **into a context**:

```
FlarisLoadBytecode(ctx, "game.flx");          /* bytecode – the shipping path */
FlarisLoadSource(ctx, "game.flx");            /* source file, compiled on load */
FlarisLoadSourceText(ctx, text, "<host>");      /* source from memory          */

FlarisRunFile("program.flx");                  /* ... and call Main(), returning its code */
```

The three loaders run the script's **top level** — global initialisers and function definitions — and stop. They do not call `Main` and do not require the script to have one. `FlarisRunFile` is the odd one out: it takes no context, is about to call `Main`, so it insists on one, and returns its exit code instead of exiting your process. It is what the `flarism` command uses.

A runtime-only build has no compiler, so `FlarisLoadBytecode` is the only one of the three it can use.

A top level runs to the end, but may not suspend

However long a global initialiser takes, it finishes: the scheduling quantum does not truncate it. What it may **not** do is suspend. A top level that calls `Fiber.Sleep`, awaits, or blocks on I/O returns `FLARIS_ERR_SUSPENDED` and is abandoned where it stopped, because the fiber it runs on is not one the scheduler can resume. Whatever it had already defined stays defined.

Do the waiting inside a function you call, not at the top level.

Load as many scripts as you like — one per context. To swap one, destroy its context and create another; the rest of the VM keeps running.

6a. Contexts

A context is one script's own global environment — a child of the VM's. What a script defines in one is invisible to every other, so two scripts that both declare `config`, or both define `Init`, no longer overwrite each other.

```
FlarisContext *game, *plugin;
FlarisCreateContext(NULL, &game);          /* NULL options = the VM's own authority */
FlarisLoadBytecode(game, "game.flx");

FlarisOptions opt = flarisOptionsDefaults;
opt.deniedModules = FLARIS_MOD_FILE | FLARIS_MOD_NET | FLARIS_MOD_OS;
```

```
opt.maxFrames      = 32;
FlarisCreateContext(&opt, &plugin);
FlarisLoadBytecode(plugin, "untrusted.flx");
```

Both scripts now run in one VM, on one thread, sharing one object pool — but not one namespace, not one set of privileges, and not one budget. For scripts that should not share a thread or a pool either, see section 12a.

What a context isolates

Names and authority. Not resources. A context is cheap — about 0.15 KB and a fraction of a microsecond to create — because it is an environment and a policy, nothing more. The object pool, the I/O slots, the fiber ceiling and the timer table are all VM-wide and shared. A context cannot be given its own heap.

Authority

`FlarisOptions` narrows what a context may do, relative to the VM's own settings:

Field	Effect
<code>grantCaps / denyCaps</code>	<code>FLARIS_CAP_*</code> , deny applied after grant
<code>deniedModules</code>	<code>FLARIS_MOD_*</code> , added to what the VM already denies

The host may grant a context more than the VM's default — it is C in your own process, and there is nothing it could not have set at `FlarisInitVM` anyway. A **script** can never widen anything: a fiber it spawns inherits its creator's authority and may only narrow from there.

Budgets

The same struct narrows what a context's fibers get to *run* with:

Field	Default	Range
<code>stackSize</code>	the VM's	64 .. the VM's
<code>maxFrames</code>	the VM's	8 .. the VM's
<code>quantum</code>	10000	any positive count of scheduling checkpoints

Unlike the capability masks, these **narrow only**: a value larger than the VM's own is rejected with `FLARIS_ERR_INIT` rather than granted, because the fiber a host call runs on is shared and was allocated once at the VM's size.

`maxFrames` is the one that decides how much an untrusted script can pin. A call frame carries the locals array, so it costs about a kilobyte: at the default of 64 a fiber holds roughly 69 KB of frames, at 1024 over a megabyte. Exceeding it raises a catchable stack exception in that context and leaves every other context untouched.

`quantum` is **advisory, not a sandbox** — `Fiber.SetQuantum` lets a script raise its own slice up to ten times the default. Use it to share a frame budget between cooperating scripts, not to contain a hostile one.

Ownership, and what dies with a context

Plain data you receive from a context — numbers, strings, arrays, objects — is yours and outlives it. Anything **callable** does not: a function from a destroyed context, or an instance of a class it defined, still exists as a

value but the environment its body needs is gone. Call into a context while it lives and let it keep its own functions.

`FlarisDestroyContext` cancels every fiber the context's code spawned and removes every timer, signal handler and file watch it registered, then releases its environment. Nothing it left behind can run afterwards.

It returns `FLARIS_ERR_BUSY`, and destroys nothing, if the context's own code is on the stack — a native module reached from one of its fibers cannot destroy the context underneath itself. Return from the call and destroy it then.

You do not have to destroy contexts at all: `FlarisShutdownVM` reclaims any you leave behind.

7. Calling into Flaris

`FlarisCall` names the callee by string, in a context:

```
int FlarisCall(FlarisContext *ctx, const char *fnName,
               const FlarisValue *args, int argc,
               FlarisValue *outResult, char *errBuf, size_t errSize);
```

It **borrow**s its arguments: you built them, you release them, and the same values may be passed to several calls. It hands back an **owned** result you release with `FlarisReleaseValue`, or `NULL` to have it released for you.

```
FlarisValue args[2] = { FlarisString("player"), FlarisInt(3) };
FlarisValue result;
char err[256];

int rc = FlarisCall(ctx, "Damage", args, 2, &result, err, sizeof err);
if (rc == FLARIS_OK) {
    printf("hp now %lld\n", (long long)FlarisAsInt(result));
    FlarisReleaseValue(result);
}
FlarisReleaseValue(args[0]);
FlarisReleaseValue(args[1]); /* a small int allocates nothing; releasing it
                             is still correct and still free */
```

Return	Meaning
FLARIS_OK	result holds the return value
FLARIS_ERR_RAISED	the script threw; err holds "code: message"
FLARIS_ERR_NOTFOUND	no such function in the loaded script
FLARIS_ERR_NOT_FN	the name exists but is not a function
FLARIS_ERR_SUSPENDED	the function suspended (see below)
FLARIS_ERR_INTERRUPTED	<code>FlarisInterrupt</code> stopped it (section 10b)
FLARIS_ERR_ARGS	bad argument count, or more than 32 arguments
FLARIS_ERR_WRONG_THREAD	ctx belongs to another VM (section 12a)

`FlarisHasFunction(ctx, "Damage")` reports whether a name is callable — useful when the script decides which hooks it implements:

```
if (FlarisHasFunction(ctx, "OnPlayerJoin")) {
    FlarisValue a = FlarisString(playerName);
    FlarisCall(ctx, "OnPlayerJoin", &a, 1, NULL, err, sizeof err);
    FlarisReleaseValue(a);
}
```

Passing `NULL` for `outResult` is fine — the return value is released for you.

Pass at most `FLARIS_MAX_CALL_ARGS` (16) arguments; a longer list is refused with `FLARIS_ERR_ARGS` rather than truncated.

Native code is used. A function the JIT compiled (the `jit` switch, on by default) runs its native code when called this way; only the argument marshalling is interpreted, the same as for a call made from script code.

Calls may nest. A native module reached from a `FlarisCall` may itself call `FlarisCall` — the boundary runs on one reserved fiber, and a nested call works above the outer one's frames and leaves them exactly as it found them. The same holds for a native module invoked while `FlarisLoadSourceText` runs a top level.

A called function must run to completion. If it yields or awaits, the call returns `FLARIS_ERR_SUSPENDED`, because nothing above it can resume it; its half-finished frames are unwound rather than left on the reserved fiber. For work that suspends, have the script start a fiber and drive `FlarisPump` instead — note that *spawning* a fiber suspends the caller too, so a function whose whole job is `Fiber.Run(...)` reports `FLARIS_ERR_SUSPENDED` even though it did exactly what you wanted. The fiber is queued either way.

8. Values and ownership

A `FlarisValue` is a one-word opaque handle to a value inside the VM. Do not dereference it, do not store it in a struct expecting to inspect it, and do not assume anything about its bit pattern. Everything you can do with one is a function call.

Making values

```
FlarisValue v;

v = FlarisNil();
v = FlarisBool(1);
v = FlarisInt(42);
v = FlarisFloat(3.5);
v = FlarisString("text");           /* NUL-terminated */
v = FlarisStringN(buf, len);        /* explicit length; may contain NULs */
v = FlarisArrayNew(16);             /* capacity hint, starts empty */
v = FlarisObjectNew();              /* string-keyed */
```

Each of these returns a value **you own**.

Reading values

Test first, then read. A read of the wrong type gives `0` or `NULL` rather than faulting, so a missing check misbehaves visibly instead of crashing.

```
if (FlarisIsInt(v))    int64_t n = FlarisAsInt(v);
if (FlarisIsFloat(v)) double  d = FlarisAsFloat(v);
if (FlarisIsBool(v))  int      b = FlarisAsBool(v);

if (FlarisIsString(v)) {
    size_t len = 0;
    const char *p = FlarisAsString(v, &len); /* borrowed, not a copy */
    fwrite(p, 1, len, stdout);
}
```

`FlarisAsString` hands back a pointer into the VM's own storage. It is valid while the value is, and it is not yours to free. Copy it if you need to keep it. It returns `NULL` for anything that is not a string — it will not coerce a number into text for you.

Arrays and objects

```
/* build */
FlarisValue a = FlarisArrayNew(3);
FlarisArrayAppend(a, FlarisInt(1)); /* append CONSUMES the value */
FlarisArrayAppend(a, FlarisInt(2));

FlarisValue o = FlarisObjectNew();
FlarisObjectSet(o, "name", FlarisString("sensor")); /* set CONSUMES too */
FlarisObjectSet(o, "id", FlarisInt(7));

/* read */
int n = FlarisArrayLen(a);
for (int i = 0; i < n; i++) {
    FlarisValue e = FlarisArrayGet(a, i); /* OWNED - release it */
    process(FlarisAsInt(e));
    FlarisReleaseValue(e);
}

FlarisValue name = FlarisObjectGet(o, "name"); /* borrowed */
```

For numeric arrays, skip the handle entirely — these allocate nothing and need no release, which matters when you are walking thousands of elements:

```
int64_t x = FlarisArrayGetInt(a, i);
double  y = FlarisArrayGetFloat(a, i);
```

And for bulk binary data, a `Block` hands you its bytes directly, so there is no per-element call at all:

```
size_t len = 0;
unsigned char *bytes = FlarisBlockData(v, &len);
```

The ownership rules

There are three, and they are the same rules in both directions.

1. What you make, you own. Every constructor above returns a value with one reference belonging to you. Release it with `FlarisReleaseValue` when you are done.

2. Some calls take it off your hands. `FlarisArrayAppend` and `FlarisObjectSet` **consume** the value you pass — the container takes your reference. Do not release it afterwards. Returning a value from a native module function (section 9) consumes it the same way: the VM takes it.

3. Borrowed values are not yours. Arguments handed to your native module function are **borrowed** — valid for that call only. If you need to keep one past the call, take your own reference:

```
FlarisValue kept = FlarisRetain(args[0]);
/* ... later ... */
FlarisReleaseValue(kept);
```

`FlarisObjectGet` also borrows. `FlarisArrayGet` does **not** — it returns an owned value, because a `[float]` array stores raw doubles and has no element object to lend you.

Scalars — `nil`, `bool`, small ints, chars — allocate nothing, so retaining and releasing them is free. You still write the calls; they simply cost nothing.

Calling by name is the only way in

`FlarisCall` hashes the name and looks it up in the context's environment on every call. There is deliberately no "resolve once and keep the callable" variant: a callable is only meaningful inside the context whose environment its body needs, so handing one back to the host would hand back something that dies when the context does.

The lookup is a hash and a probe, not a walk of a global table. Measured on the boundary suite it is well under the cost of the call itself.

9. Native modules: your C functions, called from script

Publish C functions as a module and scripts call them by name, exactly as they call a built-in namespace. Nothing is marshalled at the boundary — your function receives the same handles the script is holding.

A complete example

```
#include "flaris.h"
#include <stdio.h>

/* — the functions scripts will call — */
```

```

static double gLastOutput;

/* Device.Read(pin) -> float */
static FlarisValue DeviceRead(FlarisValue *args, int argc)
{
    (void)argc;
    int pin = (int)FlarisAsInt(args[0]);
    return FlarisFloat(read_sensor(pin));
}

/* Device.SetOutput(channel, value) -> nil */
static FlarisValue DeviceSetOutput(FlarisValue *args, int argc)
{
    (void)argc;
    gLastOutput = FlarisAsFloat(args[1]);
    write_channel((int)FlarisAsInt(args[0]), gLastOutput);
    return FlarisNil();          /* a "void" function returns nil */
}

/* Device.Status() -> object */
static FlarisValue DeviceStatus(FlarisValue *args, int argc)
{
    (void)args; (void)argc;
    FlarisValue o = FlarisObjectNew();
    FlarisObjectSet(o, "output", FlarisFloat(gLastOutput));
    FlarisObjectSet(o, "online", FlarisBool(1));
    return o;                    /* the VM takes this reference */
}

/* — the table — */
/* name      function      returns      argument types      req opt flags */
static const FlarisNative DEVICE[] = {
    {"Read",    DeviceRead,    FLARIS_T_FLOAT, {FLARIS_T_INT},    1, 0,
    FLARIS_FN_JIT_SAFE},
    {"SetOutput", DeviceSetOutput, FLARIS_T_NIL,    {FLARIS_T_INT, FLARIS_T_FLOAT}, 2, 0, 0},
    {"Status",   DeviceStatus,  FLARIS_T_OBJECT, {0},    0, 0, 0},
};

/* — wiring — */

static const char *SCRIPT =
    "fn Sample(pin: int): float {\n"
    "    let v = Device.Read(pin);\n"
    "    Device.SetOutput(1, v * 2.0);\n"
    "    return v;\n"
    "}\n";

```

```
int main(void)
{
    if (FlarisInitVM(NULL) != FLARIS_OK)
        return 1;

    /* BEFORE loading: the compiler type-checks Device.* from this table. */
    if (FlarisRegisterModule("Device", DEVICE, 3) != FLARIS_OK) {
        FlarisShutdownVM(1);
        return 1;
    }

    if (FlarisLoadSource(SCRIPT, "control.flc") != FLARIS_OK) {
        FlarisShutdownVM(1);
        return 1;
    }

    FlarisValue pin = FlarisInt(4);
    FlarisValue out;
    char err[256];

    if (FlarisCall(ctx, "Sample", &pin, 1, &out, err, sizeof err) == FLARIS_OK) {
        printf("read %.2f\n", FlarisAsFloat(out));
        FlarisReleaseValue(out);
    } else {
        printf("error: %s\n", err);
    }

    FlarisReleaseValue(pin);
    FlarisShutdownVM(0);
    return 0;
}
```

The script writes `Device.Read(pin)` the way it writes `Math.Abs(x)`. Because the compiler knows the signature you registered, passing a string where an int is declared is a **compile error**, not a runtime surprise.

Declaring the signature

`returnType` and each entry in `argTypes` is one of:

FLARIS_T_NIL	FLARIS_T_BOOL	FLARIS_T_INT	FLARIS_T_FLOAT
FLARIS_T_CHAR	FLARIS_T_STRING	FLARIS_T_ARRAY	FLARIS_T_OBJECT
FLARIS_T_BLOCK	FLARIS_T_ANY		

They are bit flags, so `FLARIS_T_INT | FLARIS_T_FLOAT` accepts either. `0` in a slot means the same as `FLARIS_T_ANY`.

`required` is how many arguments must be supplied and `optional` how many more may be. A function declaring `2, 1` accepts two or three; check `argc` yourself to see which you got. The maximum is six.

Ownership inside a native function

Two rules, both from section 8:

- **args are borrowed.** Valid for the duration of the call. To keep one, take a reference with `FlarisRetain` and release it later.
- **Your return value is consumed.** The VM takes the reference. Return `FlarisNil()` if you have nothing to give back — never a value you also released.

Making the calls fast

By default, a script function containing a call into your module is **not JIT-compiled at all** — one call the compiler cannot make discards the native code for the whole enclosing function. `FLARIS_FN_JIT_SAFE` lifts that, and it is worth a great deal in a loop:

Device.Read(i) inside a hot loop	ns per call
without the flag	136
with <code>FLARIS_FN_JIT_SAFE</code>	7.8

It is a promise the VM cannot verify, so set it only when your function:

- does not yield, await or suspend;
- does not call back into Flaris;
- does not keep a borrowed argument past the call without `FlarisRetain`;
- does not assume an interpreter frame exists.

Raising an error and having side effects are both fine. `FLARIS_FN_PURE` is a stronger claim — that the result depends only on the arguments and the call has no observable effect — which additionally lets the optimiser hoist or drop the call. Do not set it on anything reading a device, a clock or a counter.

Flag the functions your hot paths call. Flagging one function in a module whose others are unflagged still leaves any script function calling the others interpreted.

Rules and limits

- **Register before you load.** The compiler types these calls from the registry and the loader binds them; a module registered afterwards does not reach a script that is already loaded.
- **Names are matched by hash.** Two names that collide in 32 bits cannot both be registered — `FlarisRegisterModule` returns `FLARIS_ERR_COLLISION` and you rename one. A name that does not resolve when a script loads rejects that script rather than failing later at the call.
- **A module name cannot shadow a built-in** (`Math`, `Json`, `File`, ...) — that returns `FLARIS_ERR_REGISTERED`.
- **Compiled scripts become yours.** A `.flx` containing calls into your module runs only in a process that registers those exact names. That is the intended shape: these are your application's scripts, compiled when your host loads them, not artifacts meant to run under a plain `flarism`.
- **No unsafe needed.** Unlike the FFI plugin path, a native module works with the default configuration.

10. Restricting what a script can reach

By default a script can use every built-in module: the filesystem, sockets, processes, everything. A host withholds whole modules with a bitmask, one bit per module:

```
FlarisConfig cfg = flarisConfigDefaults;
cfg.deniedModules = FLARIS_MOD_FILE | FLARIS_MOD_DIRECTORY |
    FLARIS_MOD_FILEWATCH |
    FLARIS_MOD_NET | FLARIS_MOD_OS |
    FLARIS_MOD_FFI | FLARIS_MOD_BUFFER | FLARIS_MOD_MEMORY;
FlarisInitVM(&cfg);
```

The same mask on `FlarisOptions.deniedModules` denies modules for one context only, on top of whatever the VM already withholds — see section 6a.

A call into a denied module raises `Exception.ModuleDenied` (code 25), which the script can catch and your `FlarisCall` sees as `FLARIS_ERR_RAISED`. Nothing else about the script changes, and an allowed module costs nothing — the check is a bit test on an operand the call already carries.

JIT-compiled code is held to the same mask. A compiled function knows which built-in modules its native code can reach, counting every compiled function it calls; in a context that denies one of them it runs interpreted, so the call raises exactly where it would with the JIT off. That covers fibers the context spawns and callbacks a built-in runs for it (`Array.Map`, `Memory.Process`, ...). A library imported by both a trusted and a sandboxed context shares one compiled copy, used natively by the first and interpreted by the second.

The constants are `FLARIS_MOD_` plus the module name in upper case: `FLARIS_MOD_FILE`, `FLARIS_MOD_NET`, `FLARIS_MOD_OS`, `FLARIS_MOD_JSON` and so on, one for each of the 32 built-in modules. They are positional, so recompile against the header you ship with.

TLS is not separately gated. A client speaking `https://` is doing what `Net` and `Stream` are for; if a script should not reach the network, deny those modules. What does need its own gate is a script's ability to **load more code**, which is what the `FLARIS_CAP_IMPORT_*` capabilities in section 5 are for.

What this is, and what it is not

It is **refusal, not confinement**. You can remove the filesystem; you cannot grant a subdirectory. There is no path allow-list, no host-supplied resolver and no per-call hook.

The unit is the **whole module**. Denying `FLARIS_MOD_OS` removes `Os.Execute` and `Os.Spawn`, and also `Os.Args` and `Os.Arch`, which only report the environment. It errs towards denying more.

And it bounds **capability, not consumption**. A script with every module denied can still loop forever.

`FlarisInterrupt` (section 10b) is what stops it, and it is the piece you have to wire up yourself — a deadline, a step budget, a signal handler; the VM has no timeout of its own.

Per-context budgets (section 6a) do not close this either. `maxFrames` bounds recursion depth, and `quantum` bounds how long a *fiber* holds the scheduler before the next one runs — neither bounds a straight loop inside a single call. Memory is the gap that remains: allocation is pooled VM-wide rather than charged per context,

so a script that allocates without bound reaches the VM's pool limit rather than a limit of its own. If you are running code you do not trust at all, that is still the reason to run it in a process you can kill.

10a. Signals from the script

`Vm.NotifyHost(code)` hands a code to you, and does nothing if you are not listening:

```
static void OnNotify(int code, FlarisValue payload, void *userData)
{
    (void)userData;
    if (code == 7 && FlarisIsString(payload))
        SetStatusText(FlarisAsString(payload, NULL));
}

FlarisSetNotifyHandler(OnNotify, NULL);
```

```
fn Rebuild(items: [int]) {
    for (let i: int = 0; i < items.Length; i++) {
        Step(items[i]);
        VM.NotifyHost(7, "rebuilding " + str(i));
    }
}
```

The payload is optional and may be any value; it is **borrowed**, so retain it if you keep it. The call returns `true` when a handler ran and `false` when none is installed.

That last part is the point of it. A script calling `Vm.NotifyHost` still compiles and runs under a host that ignores it, and under the plain `flarism` command — where it simply returns `false`. A native module cannot do that: a script calling `Host.Progress(...)` will not compile unless the module exists. Use a native module when you want types and a real name; use this when you want a channel that is always there.

It is cooperative. It fires only where the script chose to call it, so it is not a way to stop a script that never returns. `FlarisInterrupt` is — see the next section.

10b. Stopping a running script

```
void FlarisInterrupt(FlarisContext *ctx);
```

Stops whatever `ctx` is running. Every fiber that context owns unwinds as soon as it reaches a checkpoint: its `finally` blocks run, no `catch` in the script is offered the exception, and a call in progress returns `FLARIS_ERR_INTERRUPTED` with the reason in `errBuf`.

This is the piece that makes an untrusted script bounded in time. The VM has no timeout of its own, deliberately — a deadline that suits a game frame does not suit a batch job — so you set the policy and call this when it is exceeded:

```
// A watchdog thread. FlarisInterrupt is one store to a flag, which is why it
// may be called from a thread that is not the VM's, and from a signal handler.
static void *Watchdog(void *arg)
{
    Deadline *d = arg;
    while (!Elapsed(d))
        Nap(5);
    FlarisInterrupt(d->ctx);
    return NULL;
}

int rc = FlarisCall(ctx, "Render", args, 2, &out, err, sizeof err);
if (rc == FLARIS_ERR_INTERRUPTED)
    Log("script exceeded its budget: %s", err);
```

The same call works from inside one of your own native module functions, which is how a step budget is written without a second thread: count the calls, and interrupt when the count is exceeded.

It reaches a loop that yields nothing. The check rides on the same per-instruction checkpoint the scheduler uses, so an endless `while (true)` stops within a few thousand instructions — including inside a `FlarisCall`, where the scheduler itself is switched off.

What it does not cut short is a single native call. If your own module blocks for a minute, it blocks for a minute; the interrupt is taken when control returns to the interpreter. The same is true of a **JIT-compiled** loop, which runs as one native call with no checkpoints in it — set `jit = FLARIS_OFF` on any context whose scripts have to stay interruptible.

The script cannot refuse. `catch` never sees it, and the exception is re-raised past every handler until the fiber ends. `finally` still runs, so a script that holds a file or a lock releases it on the way out.

The request is latched. It stays set until your next `FlarisCall` or `FlarisLoad*` on that context, so every fiber still queued in it stops too, not only the one that was running. `FlarisPump` deliberately does *not* clear it — that is what lets you pump the interrupted fibers to their end:

```
FlarisInterrupt(ctx);
while (FlarisHasWork(ctx))
    FlarisPump(ctx, 0); // their finally blocks run here
```

Safe to call when nothing is running, and safe to call repeatedly. A context that was interrupted is not damaged: the next call runs normally.

10c. Embedding from C#

`libflaris` is a plain C shared library with 51 exported symbols, all prefixed `Flaris`, and no other public surface. That makes it a direct P/Invoke target — no C++ name mangling, no wrapper layer, no generated glue.

Everything in this guide works from C#, including native modules whose functions are C# methods.

The pieces that need care

FlarisValue is one word. Wrap it in a `readonly struct` over `nuint` so the compiler keeps handles apart from integers:

```
public readonly struct FlarisValue(nuint raw) { public readonly nuint Raw = raw; }
```

Use `[LibraryImport]`, not `[DllImport]`. It generates the marshalling at compile time, and `StringMarshalling.Utf8` gives you the UTF-8 the API expects:

```
[LibraryImport("flaris", StringMarshalling = StringMarshalling.Utf8)]
public static partial int FlarisLoadSourceText(IntPtr ctx, string source, string name);
```

Callbacks must be `[UnmanagedCallersOnly]`, not delegates. A delegate needs the GC to keep it alive and cannot be called from JIT-compiled Flaris code. An `[UnmanagedCallersOnly]` static method gives you a real function pointer:

```
[UnmanagedCallersOnly]
static FlarisValue DeviceRead(FlarisValue* args, int argc)
    => FlarisFloat(FlarisAsInt(args[0]) * 1.5);
```

The two structs are sequential and blittable. `FlarisNative` holds a fixed-size array, so it needs `unsafe` and a `fixed` buffer:

```
[StructLayout(LayoutKind.Sequential)]
public unsafe struct FlarisNative
{
    public IntPtr Name; // 0
    public delegate* unmanaged<FlarisValue*, int, FlarisValue> Fn; // 8
    public uint ReturnType; // 16
    public fixed uint ArgTypes[6]; // 20
    public byte Required, Optional; // 44, 45
    public uint Flags; // 48
} // 56 bytes
```

`FlarisConfig` is 80 bytes and `FlarisOptions` 32, and neither needs attributes beyond `LayoutKind.Sequential` — every field is an `int`, `uint`, `ushort`, `ulong` or pointer, in declaration order.

A `FlarisContext *` is just an opaque pointer. Hold it as `IntPtr`; every loading and calling entry point takes one as its first argument.

Registering a module of C# functions

```
var fns = stackalloc FlarisNative[1];
fns[0].Name = Marshal.StringToHGlobalAnsi("Read");
```

```
fns[0].Fn          = &DeviceRead;
fns[0].ReturnType = T.Float;
fns[0].ArgTypes[0] = T.Int;
fns[0].Required   = 1;
fns[0].Flags      = FlarisNative.JitSafe;

FlarisRegisterModule("Device", fns, 1);

FlarisCreateContext(IntPtr.Zero, out IntPtr ctx);
FlarisLoadSourceText(ctx, "fn Sample(p: int): float { return Device.Read(p); }", "s");
```

The script calls `Device.Read(p)` and your C# method runs. The name pointer must outlive registration — allocate it unmanaged, or pin it — but the VM copies the table itself, so `stackalloc` for the array is fine.

What is verified to work

A C# host built this way was checked against the library: initialising with a denied-module mask, registering a module of C# methods, calling scripts by name, strings and arrays round-tripping in both directions, reading an object's keys, constructing a script class from C# and calling a bound method on the instance, receiving `Vm.NotifyHost`, and seeing a script exception arrive as a return code rather than an unwind through native frames.

The constraint that matters most

The threading rule in section 12a applies: **every call must come from the thread that called `FlarisInitVM`** for that VM. In C# that means no `async` continuation that might resume elsewhere, and no `Task.Run` around a call. If your application is `async`, marshal each VM's work onto a single dedicated thread and keep it there.

Note also that a native module function running on the VM thread must not block for long: the scheduler is cooperative, so a C# method that waits stalls every fiber.

11. Driving the scheduler

Scripts that use timers, fibers or `async` I/O need scheduler time. You choose who owns the loop.

Your loop owns it. `FlarisPump` runs one non-blocking pass and returns how many fibers ran. It **never sleeps**, so your application decides its own cadence:

```
while (running) {
    FlarisPump(NULL, 0);    /* NULL = every context; 0 = every fiber ready now */
    RenderFrame();
}
```

Bound the work per frame so a busy script cannot stall you:

```
FlarisPump(NULL, 4);      /* at most four fibers this frame */
```

Pass a context instead of `NULL` to decide *whose* work advances. Only the fibers that context spawned get CPU; every other context's stay queued exactly where they were:

```
FlarisPump(untrusted, 2);    /* the plugin gets two fibers' worth of this frame */
FlarisPump(engine, 0);      /* your own scripts run to the end of their queue */
```

One thing stays VM-wide either way: signals, I/O completions and due timers are delivered on every pump, because otherwise this context's own I/O would never be reaped. So a file watch or an async completion belonging to a *different* context can still fire during a per-context pump. What the context bounds is which fibers run, not which completions arrive.

The VM owns it. `FlarisRunToCompletion` runs until nothing can become ready again — no queued fiber, no live timer, no claimed event, no in-flight I/O — sleeping while fibers wait rather than spinning:

```
FlarisCall(ctx, "Start", NULL, 0, NULL, err, sizeof err);
FlarisRunToCompletion();
```

It is the only entry point in the library that blocks, and it blocks for as long as the scripts keep working: a script that never finishes never returns. It is VM-wide by nature — it has to wait on every context's I/O to know that none of them can make progress.

`FlarisHasWork(NULL)` is true while a queued fiber, a live timer, a claimed event or an in-flight I/O operation could still make progress — the termination condition for a loop of your own:

```
while (FlarisHasWork(NULL)) {
    if (FlarisPump(NULL, 0) == 0)
        SleepMilliseconds(1);    /* nothing was ready - your call how to idle */
}
```

`FlarisHasWork(ctx)` answers the same question about one context: an unfinished fiber it owns — queued, sleeping or parked on I/O — or a timer it armed. That answer costs a walk of the live-fiber registry where the VM-wide form is a few counter reads, so call it once per loop iteration rather than per fiber.

Because `FlarisPump` never sleeps, a loop with no idle of its own can spin thousands of times before a 1 ms timer comes due. That is deliberate: the VM does not decide how long your application blocks.

12. Errors and diagnostics

Script errors come back as values

A script that throws does not print and vanish. The call returns `FLARIS_ERR_RAISED` and writes the exception's code and message into your buffer:

```
char err[256];
int rc = FlarisCall(ctx, "Risky", NULL, 0, &result, err, sizeof err);
```

```
if (rc == FLARIS_ERR_RAISED)
    LogWarning("script error: %s", err);    /* e.g. "7: inventory full" */
```

`err` is always NUL-terminated and is cleared on entry, so a stale message never survives into a later call. The VM stays usable afterwards — the next call runs normally.

Passing `NULL` (with size `0`) for the buffer is supported and skips formatting the message entirely. `rc` still tells you the call raised, so a host that only branches on failure and never shows the text can use it:

```
int rc = FlarisCall(ctx, "Risky", NULL, 0, &result, NULL, 0); /* no message built */
```

That is worth doing only where raises are frequent enough to matter — building the message costs roughly a tenth of a microsecond per raised call. Where you report the error to a human or a log, take the buffer.

This holds for *any* failure inside the call, not just an explicit `throw`: a division by zero, an index out of bounds, or a blocked `Ffi` call with `unsafe` off all arrive the same way.

Nothing takes your process down

Nothing a script does, and nothing an input file contains, ends your process. An unhandled error is reported and unwound, a corrupt `.flx` is refused however deep the damage lies, a module whose pinned hash does not match is refused, a bundled dependency that fails the `requireSigned` check is withheld, and a module that fails to execute is refused — each as a return value, with the VM usable afterwards. That is what differs from the `flarism` command, which exits on all of these because exiting is the right thing for a command to do.

Two script-level constructs change meaning accordingly, because a script's decision to stop is not a decision that your application should stop:

In the script	Under <code>flarism</code>	Under your host
<code>Vm.Exit(code)</code>	exits with <code>code</code>	the call returns <code>FLARIS_ERR_RAISED</code> , naming the code
<code>Debug.AssertTrue / AssertEqual</code> failing exits		raises, after printing the trace

Two things are outside that guarantee, and both are yours to control:

- **Allocator exhaustion is fatal only as a last resort.** When the object pool hits `cfg.maxSlabs`, the allocation raises `Exception.OutOfMemory` out of a small reserve kept for exactly that, so the script (and your `FlarisCall`) sees an ordinary raise. Only when that reserve is itself spent, or the system refuses memory outright, does the VM report and exit. Size `maxSlabs` for your workload; the default ceiling is 1024 slabs.
- **The signal handlers are opt-out.** With `installSignals` left on, a `SIGINT`, `SIGTERM` or `SIGHUP` that no script handler has claimed ends the process immediately (exit code 128 + signal number) with nothing torn down. Set `cfg.installSignals = FLARIS_OFF` to keep your own. The VM never handles `SIGSEGV`: a fault inside it reaches your handler, or the default. `FlarisShutdownVM` puts the dispositions it replaced back, so a signal after shutdown reaches whatever handler you had before — the VM's handler never runs against a torn-down VM. `SIGPIPE` is the exception: the VM leaves it ignored, since a stray write to a closed peer should not kill your process either.

Routing the VM's own output

By default the VM writes its diagnostics — runtime errors, warnings, compiler messages — to `stderr`. Hand it a callback instead:

```
static void MyLogger(int level, const char *message, void *userData)
{
    static const char *names[] = { "error", "warning", "info", "verbose" };
    fprintf(myLogFile, "[flaris %s] %s\n", names[level], message);
}

FlarisSetLogHandler(MyLogger, NULL);
```

`message` is formatted, NUL-terminated, and carries no trailing newline and no ANSI escapes; it is valid only for the duration of the call, so copy it if you keep it. Levels are `FLARIS_LOG_ERROR`, `FLARIS_LOG_WARNING`, `FLARIS_LOG_INFO` and `FLARIS_LOG_VERBOSE`. Pass `NULL` to go back to `stderr`.

You may set the handler before `FlarisInitVM`, so even startup diagnostics reach it. Compilation diagnostics go through it too, which is how you capture why a script failed to compile.

This covers what the *VM* says. What a *script* prints with `Console.WriteLine` still goes to `stdout`; redirect that by your own means if you need to.

12a. Threads: one VM per thread

Everything up to here has described one VM. A host that wants parallelism starts more: **every thread that calls `FlarisInitVM` gets a VM of its own**, and any number of threads may do so.

No function here takes a VM argument, because each one acts on the VM of the thread that calls it. The code that runs a single VM runs one per thread unchanged:

```
static void *Worker(void *arg)
{
    FlarisConfig cfg = flarisConfigDefaults;
    cfg.installSignals = FLARIS_OFF; /* let one thread own the process's signals */
    cfg.ioThreads      = 2;          /* this VM's own I/O workers */
    FlarisInitVM(&cfg);

    FlarisContext *ctx;
    FlarisCreateContext(NULL, &ctx);
    FlarisLoadSource(ctx, (const char *)arg);

    char err[256];
    FlarisCall(ctx, "Run", NULL, 0, NULL, err, sizeof err);
    while (FlarisHasWork(NULL))
        FlarisPump(NULL, 0);
}
```

```

    FlarisDestroyContext(ctx);
    FlarisShutdownVM(0);
    return NULL;
}

```

What is isolated

Everything that makes a VM a VM. Two VMs share no objects, no globals, no classes, no fibers, no timers and no I/O workers, and nothing between them is locked on the calling path.

Native modules are per VM too, which is the one thing that catches people out: call `FlarisRegisterModule` on **each** thread, after that thread's `FlarisInitVM`. Registering on one thread does not publish the module to the others.

Values and contexts do not travel either. A `FlarisValue` belongs to the VM that made it and means nothing to another. A context belongs to the VM that created it, and handing one to the wrong thread returns `FLARIS_ERR_WRONG_THREAD` from every entry point that takes a context, rather than corrupting either side. Passing a `FlarisValue` across is *not* detected — copy what you need over as plain C data.

`FlarisInterrupt` is the deliberate exception: it is one store to a flag, so a watchdog thread may aim it at any context (section 10b).

What is still shared, because the process has only one

Shared	What it means for you
Signal dispositions	The first VM configured with <code>installSignals</code> on installs them; they are restored when the last such VM shuts down. Leave it on for one thread, or off everywhere and handle signals yourself.
Compilation	Serialized across the process: two threads compiling at the same moment take turns. Compile once, or ship bytecode, rather than loading source in a hot loop on many threads.
Libraries loaded through <code>Ffi</code>	The mapping is shared and stays loaded while any VM holds it, so the plugin's own code may run on several threads at once and its globals must be thread-safe. Callback registrations are <i>not</i> shared — a name each VM registers resolves only for that VM.
<code>FlarisSetArgs</code>	One <code>argv</code> for the process; the last caller wins for every VM.
Environment and working directory	<code>Os.SetEnv</code> and <code>Os.Chdir</code> change them for the whole process: every VM, and every child process started afterwards, sees the change. <code>Os</code> serialises its own environment reads and writes and its process launches across threads; a native module or FFI plugin that calls <code>getenv</code> / <code>setenv</code> itself is outside that lock. Child processes themselves are per VM — <code>Os.KillChild</code> and the <code>Os</code> pipe calls accept only what that VM's <code>Os.Spawn</code> returned.
<code>stdout</code> and <code>stderr</code>	Output from several VMs interleaves. A log handler is per VM (section 12), so set one on each thread to keep the VM's own diagnostics apart; a script's own printing is yours to redirect.
The process	A native module or an FFI plugin that crashes takes every VM with it.

Threads or contexts?

They answer different questions, and they compose.

- A **context** isolates names, authority and budgets inside one VM, on one thread (section 6a). It costs about 0.15 KB and a fraction of a microsecond, so it is the right tool for running several scripts that must not see each other.
- A **thread with its own VM** isolates everything else as well, adds parallelism, and gives that script its own `maxSlabs` ceiling. It costs a whole VM: its own object pool and its own workers.

Untrusted code is a context question. Throughput, and a hard memory ceiling per script, are thread questions.

13. Limits and what is not supported

One VM per thread. A thread holds at most one VM and there is no handle type, so every call acts on the calling thread's VM. Several scripts inside one VM are contexts (section 6a); several VMs are threads (section 12a). Neither a context nor a value crosses from one VM to another.

Fibers do not span threads. Fibers are cooperative and run on their own VM's thread, so `Fiber.Run` buys concurrency, not parallelism. For parallelism, start a second VM on a second thread.

A native module is bound to the host that registers it. Section 9 covers this in full: a `.flx` containing calls into your module runs only in a process that registers those exact names, and the standalone `flarism --compile` cannot compile a script that calls one, because it does not know your module. Compile such scripts from your own host, at load.

No memory budget per context. Allocations come from a VM-wide pool, so one context cannot be given its own ceiling; a script that allocates without bound exhausts the VM's pool, which raises rather than aborting, but does so for everyone in that VM. A script that must have its own ceiling needs its own VM on its own thread, where `maxSlabs` bounds it alone (section 12a). Recursion depth and scheduler share are bounded per context (section 6a), and a runaway loop is stoppable (section 10b); memory is not yet.

A JIT-compiled loop cannot be stopped. Native code runs as one C call with no checkpoints in it, so neither the scheduler nor `FlarisInterrupt` reaches a loop inside it. Set `jit = FLARIS_OFF` for any context whose scripts must stay interruptible.

A malformed `.flx` is refused, not survived halfway. The bytecode reader abandons the whole load on the first inconsistency — `FlarisLoadBytecode` returns `FLARIS_ERR_COMPILE` and nothing from that file is defined. It does not attempt partial recovery, so a damaged file is never half-loaded.

An **FFI plugin** remains the alternative when the code you want to reach lives in a shared library you do not compile against: the script loads and calls it through `Ffi`, which requires `FLARIS_CAP_FFI` (or `FLARIS_CAP_UNSAFE`, which implies it) and is refused outright when `FLARIS_MOD_FFI` is denied. A native module needs neither.

14. API reference

Everything below is declared in `flaris.h`.

Lifecycle

Function	Description
<code>int FlarisInitVM(const FlarisConfig *cfg)</code>	Start the VM. <code>NULL</code> means defaults. <code>FLARIS_OK</code> or <code>FLARIS_ERR_INIT</code> .
<code>void FlarisShutdownVM(int code)</code>	Release everything; idempotent. <code>code</code> only reaches an attached debugger.
<code>void FlarisSetArgs(int argc, char **argv)</code>	Make <code>argv</code> visible to <code>Os.Args</code> . Borrows <code>argv</code> .

Function	Description
<code>void FlarisSetLogHandler(FlarisLogFn fn, void *userData)</code>	Route VM diagnostics to <code>fn</code> ; <code>NULL</code> restores <code>stderr</code> .

Contexts

Function	Description
<code>int FlarisCreateContext(const FlarisOptions *opt, FlarisContext **out)</code>	Create one. <code>NULL</code> options means the VM's own authority.
<code>int FlarisDestroyContext(FlarisContext *ctx)</code>	Cancel its fibers, drop its timers/watches/handlers, release it. <code>FLARIS_ERR_BUSY</code> from inside its own code.
<code>const FlarisOptions flarisOptionsDefaults</code>	All-zero: the VM's authority and budgets.

Loading

Function	Description
<code>int FlarisLoadSourceText(ctx, const char *source, const char *name)</code>	Compile a string into <code>ctx</code> and run its top level. <code>name</code> labels diagnostics.
<code>int FlarisLoadSource(ctx, const char *path)</code>	The same, reading <code>.fls</code> from a path. Not in a runtime-only build.
<code>int FlarisLoadBytecode(ctx, const char *path)</code>	Load <code>.flx</code> into <code>ctx</code> and run its top level.
<code>int FlarisRunFile(const char *path)</code>	No context: load and call <code>Main()</code> , returning its exit code.

Calling

Function	Description
<code>int FlarisCall(ctx, name, args, argc, outResult, errBuf, errSize)</code>	Call a function in <code>ctx</code> by name. Borrows <code>args</code> ; the result is owned.
<code>int FlarisHasFunction(ctx, const char *name)</code>	1 if the name is callable in <code>ctx</code> .

Signals from the script

Function	Description
<code>void FlarisSetNotifyHandler(FlarisNotifyFn fn, void *userData)</code>	Listen for <code>Vm.NotifyHost(code[, payload])</code> . <code>NULL</code> stops listening.

Sandboxing

Field	Description
<code>uint64_t FlarisConfig.deniedModules</code>	Bitmask of <code>FLARIS_MOD_*</code> . 0 (the default) allows everything.
<code>uint32_t FlarisConfig.grantCaps / .denyCaps</code>	<code>FLARIS_CAP_*</code> ; deny is applied after grant.
<code>uint64_t FlarisOptions.deniedModules</code>	Per context, added to what the VM denies.
<code>uint32_t FlarisOptions.grantCaps / .denyCaps</code>	Per context, relative to the VM's authority.
<code>uint32_t FlarisOptions.stackSize / uint16_t .maxFrames / int32_t .quantum</code>	Per-context budgets. Narrow only — a larger value is rejected.

Native modules

Function	Description
<code>int FlarisRegisterModule(name, fns, count)</code>	Publish C functions as the module <code>name</code> . Call before loading.
<code>FLARIS_FN_JIT_SAFE</code>	Flag: native code may call this function directly. Roughly 3× faster; see the promise above.
<code>FLARIS_FN_PURE</code>	Flag: no side effects, result depends only on the arguments. Implies JIT-safe.

Values

`FlarisValue` is an opaque handle to a value inside the VM. Constructors return an owned value; readers return borrowed data valid while the value is.

Function	Description
<code>FlarisNil()</code> / <code>FlarisBool(v)</code> / <code>FlarisInt(v)</code> / <code>FlarisFloat(v)</code>	Scalars.
<code>FlarisString(s)</code> / <code>FlarisStringN(s, len)</code>	Strings.
<code>FlarisIsNil(v)</code> / <code>FlarisIsBool</code> / <code>FlarisIsInt</code> / <code>FlarisIsFloat</code>	Scalar type tests.
<code>FlarisIsString(v)</code> / <code>FlarisIsArray(v)</code> / <code>FlarisIsObject(v)</code>	Container type tests.
<code>FlarisIsInstance(v)</code> / <code>FlarisClassName(v)</code>	A class instance, and its class name.
<code>FlarisAsInt(v)</code> / <code>FlarisAsFloat(v)</code> / <code>FlarisAsBool(v)</code>	Numeric reads; 0 on a type mismatch.
<code>const char *FlarisAsString(v, &len)</code>	Borrowed chars, NULL for a non-string.
<code>void *FlarisBlockData(v, &len)</code>	A Block's bytes directly — bulk data with no per-element call.
<code>FlarisArrayNew(cap)</code> / <code>FlarisArrayLen(a)</code> / <code>FlarisArrayAppend(a, v)</code>	Arrays; append consumes <code>v</code> .
<code>FlarisArrayGet(a, i)</code>	Owned element — release it.
<code>FlarisArrayGetInt(a, i)</code> / <code>FlarisArrayGetFloat(a, i)</code>	Numeric elements without boxing.
<code>FlarisObjectNew()</code> / <code>FlarisObjectLen(o)</code> / <code>FlarisObjectGet(o, key)</code>	Objects; <code>Get</code> borrows.
<code>FlarisObjectSet(o, key, v)</code>	Consumes <code>v</code> . Works on instances too.
<code>FlarisObjectNext(o, &cursor, &key, &keyLen, &value)</code>	Walk every property in insertion order. Both borrowed.
<code>FlarisRetain(v)</code> / <code>FlarisReleaseValue(v)</code>	Keep a borrowed value / drop an owned one.

Scheduler

Function	Description
<code>int FlarisPump(FlarisContext *ctx, int maxFibers)</code>	One non-blocking pass; returns fibers run. Never sleeps. <code>ctx</code> NULL drives every context.
<code>void FlarisInterrupt(FlarisContext *ctx)</code>	Stop what <code>ctx</code> is running: its fibers unwind through their <code>finally</code> blocks and a call in progress returns <code>FLARIS_ERR_INTERRUPTED</code> . One store to a flag, so it is safe from a signal handler or another thread.
<code>int FlarisHasWork(FlarisContext *ctx)</code>	True while anything could still make progress; <code>ctx</code> NULL asks about the whole VM.
<code>void FlarisRunToCompletion(void)</code>	Run until quiescent, sleeping as needed. Blocks; VM-wide.

Status codes

Constant	Value	Meaning
FLARIS_OK	0	Success
FLARIS_ERR_NO_VM	-1	No VM is running
FLARIS_ERR_NOT_FN	-2	The name exists but is not callable
FLARIS_ERR_ARGS	-3	Bad argument count
FLARIS_ERR_RAISED	-4	The script threw
FLARIS_ERR_SUSPENDED	-5	The function yielded or awaited
FLARIS_ERR_INIT	-10	A configuration value was rejected
FLARIS_ERR_COMPILE	-11	Script did not compile
FLARIS_ERR_NOTFOUND	-12	No such function
FLARIS_ERR_REGISTERED	-13	That module name is already taken
FLARIS_ERR_COLLISION	-14	Two names share a 32-bit hash; rename one
FLARIS_ERR_BUSY	-15	The context is running its own code; it cannot be destroyed now
FLARIS_ERR_INTERRUPTED	-16	<code>FlarisInterrupt</code> stopped the call
FLARIS_ERR_WRONG_THREAD	-17	That context belongs to another VM (section 12a)